Solidity Code Explanation

// // 1. We tell the compiler what license we're using. It's just a good habit.
// // SPDX-License-Identifier: MIT
// // 2. We tell the compiler which version of Solidity to use.
// pragma solidity ^0.8.20;
// // 3. We define our contract. Think of it like a 'class' in C++.
// contract HelloWorld {
//   // 4. This is a "State Variable". It is data stored permanently
//   //   with the contract on the blockchain.
//   string public greeting = "Hello, World!";
//   uint public myNumber = 30;// take 256 bit storage (unsigned integer)
// }

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract MyFavoriteNumber {
uint public myNumber = 10;
// This is our new function!
function setMyNumber(uint _newNumber) public {
myNumber = _newNumber;
}
} explain this complete code in details each line and with more examples

Alright! Let’s break down your Solidity code step by step, line by line, in detail — and I’ll add some practical explanations and extra examples to make it super clear for you.

📜 Your Code

```
solidity


// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyFavoriteNumber {
    uint public myNumber = 10;

    function setMyNumber(uint _newNumber) public {
        myNumber = _newNumber;
    }
}
```

✅ Detailed Explanation

1️⃣ `// SPDX-License-Identifier: MIT`

What is it?
This line tells the compiler and anyone reading the code what license you're using.

- `SPDX` stands for **Software Package Data Exchange**.
- `MIT` means you're using the **MIT License**, which is very permissive — anyone can reuse, modify, and share your code.

Why?
It's good practice and some compilers (like those used on Etherscan) require it.

2️⃣ `pragma solidity ^0.8.20;`

What is it?
This tells the compiler which version of Solidity your code is written for.

- `pragma` means "compiler instruction".
- `^0.8.20` means **version 0.8.20 or higher, but less than 0.9.0**.

Why?
Solidity changes over time — new features, bug fixes, and breaking changes. Locking the version avoids unexpected errors.

3️⃣ `contract MyFavoriteNumber {`

What is it?
This defines a **contract**, which is like a class in other programming languages (e.g., C++, Java).
A **contract** in Solidity is the main building block. It contains:

- State variables
- Functions
- Events
- Structs, Enums, Mappings, etc.

4️⃣ `uint public myNumber = 10;`

What is it?

- `uint` is an **unsigned integer** type.
  - It defaults to `uint256` (256 bits).
- `public` makes this variable **accessible to anyone** outside the contract.
  - Solidity automatically creates a **getter function** for you.
- `myNumber` is the name of the variable.

- `= 10;` initializes it with **10**.

Where is it stored?
It's a **state variable**. It lives **permanently** on the blockchain.

5️⃣ `function setMyNumber(uint _newNumber) public {`

What is it?
This defines a **function** you can call to **change the value** of `myNumber`.

- `function` declares a function.
- `setMyNumber` is the name of the function.
- `(uint _newNumber)` means it takes **one argument**, which must be a `uint`.
  - The _ prefix is just a convention to show it's an input.
- `public` means **anyone can call this function**.

6️⃣ `myNumber = _newNumber;`

What is it?
This line **assigns** the new value to `myNumber`.

When you call `setMyNumber(42)`, this sets `myNumber` to `42` permanently on the blockchain.

7️⃣ Closing Braces `}`

They close the function and the contract.

📌 How it works

1. **Deploy the contract** → `myNumber` starts at `10`.
2. Anyone can **read** `myNumber` for free (because `public` makes a getter).
3. Anyone can **call** `setMyNumber()` to change its value.
   - This **costs gas** because it **modifies the blockchain state**.

🧩 More Practical Examples

Let's see a few **small changes** you can try:

✅ Example 1: Add another function to double the number

```
solidity

function doubleMyNumber() public {
    myNumber = myNumber * 2;
```

```
}
```

- This function takes no arguments.
- It multiplies the current `myNumber` by 2.

✅ Example 2: Add a getter manually

```
solidity

function getMyNumber() public view returns (uint) {
    return myNumber;
}
```

- This does the same as the `public` variable's auto-getter.
- `view` means it **does not change** the blockchain.
- `returns (uint)` means it **returns** a `uint`.

✅ Example 3: Use an `address` type

```
solidity

address public owner;

constructor() {
    owner = msg.sender; // sets the owner to the address that deploys the contract
}
```

- `address` stores an Ethereum wallet address.
- `constructor()` runs **only once** when the contract is deployed.

✅ Example 4: Add a `require` check

```
solidity

function setMyNumber(uint _newNumber) public {
    require(_newNumber > 0, "Number must be greater than zero");
    myNumber = _newNumber;
}
```

- `require` is like an `if` statement that **reverts** if the condition is false.

✅ Example 5: Add an `event`

```
solidity

event NumberChanged(uint newNumber);

function setMyNumber(uint _newNumber) public {
    myNumber = _newNumber;
```

```
    emit NumberChanged(_newNumber);
}
```

- `event` logs information to the blockchain.
- `emit` creates the log.
- Useful for frontend apps to listen for changes.

🔑 Key Takeaways

✅ `pragma` = compiler version
✅ `contract` = like a class
✅ `state variable` = stored on blockchain
✅ `public` = anyone can see/call
✅ `function` = does something (can read or write)
✅ `set` = changes state (costs gas)
✅ `view` = reads state (free)

🌐 Summary

This is a **simple smart contract** that:

- Stores a number (`myNumber`)
- Lets anyone **read** it
- Lets anyone **change** it

It's a good starting point to learn how Solidity handles **variables, functions, access control, and storage**.

If you'd like, I can write:
✅ A **more advanced version** (e.g., only owner can change the number)
✅ An example with **arrays, mappings, or structs**
✅ A **step-by-step guide** to deploy it on Remix or Hardhat

👉 Just tell me! Want to expand this?